Module 2 PINN Implementation Note

Detailed How-To Guide for `RadiationEffects_proj2`

Running, testing, inspecting, tuning, and diagnosing the two-dimensional electrostatic physics-informed neural network

---

Purpose and scope. This is the operational companion to the Module 2 PINN physics note and the Module 2 FEM implementation note. It documents the executable MATLAB PINN path beginning at `main_module2_pinn_electrostatics.m`: required software, exact commands, the seven-file call chain, every user option, nondimensionalization, automatic differentiation, loss construction, returned data, verification plots, tests, diagnostic probes, and common failures. The objective is that a reader can first establish that the reference FEM solver is healthy, then run the smallest PINN smoke test, then perform scientifically meaningful analytical and FEM comparisons without guessing what any function does.

Architecture migration notice (2026-08-19). The project-level Module 2 contract has changed to field-to-field learning, $\boldsymbol{\rho} \mapsto \boldsymbol{\Phi}_\theta$. The FEM/data pipeline already accepts arbitrary nodal charge fields and writes v2 complete-field datasets. The executable files documented in this implementation note still implement the older pointwise single-case PINN, $(x, y) \mapsto \phi_\theta(x, y)$, and are retained as regression/prototype code until the neural implementation is migrated. Therefore this document is authoritative for how the *current legacy PINN code* runs, while the revised `module2_PINN.pdf` physics note is authoritative for the new target architecture.

Fastest safe start for the legacy prototype. Open MATLAB in `RadiationEffects_proj2/matlab/`, run `setup_project_paths`, confirm the Deep Learning Toolbox is available, run `test_module2_fem_all_2d`, and then run `test_module2_pinn_electrostatics`. After those preflight checks, train the analytical linear-potential case before the Gaussian defect case.

Contents

Symbols and acronyms used in this document

Symbol / acronym	Name	Meaning in this document
PINN	Physics-informed neural network	Neural representation trained using the governing PDE, boundary conditions, and optional FEM anchors.

Symbol / acronym	Name	Meaning in this document
MLP	Multilayer perceptron	Fully connected feed-forward network used by this implementation.
FEM	Finite-element method	Reference Module 2 solver used to construct anchors and validate the trained PINN.
AD	Automatic differentiation	MATLAB differentiation of network outputs with respect to coordinates and trainable parameters.
Ω	Computational domain	Rectangle $[0, L_x] \times [0, L_y]$.
ϕ	Electrostatic potential	Physical potential in volts.
ϕ_θ	Network potential	PINN approximation to ϕ.
$\mathbf{E}$	Electric field	$\mathbf{E} = -\nabla\phi$, in volts per meter.
ρ	Space-charge density	Physical source in coulombs per cubic meter.
$\varepsilon_{\mathrm{Si}}$	Silicon permittivity	Constant scalar permittivity used by the current FEM and PINN paths.
$\hat{x}, \hat{y}$	Normalized coordinates	$\hat{x} = x/L_x$ and $\hat{y} = y/L_y$.
$\hat{\phi}$	Normalized potential	Network output $\hat{\phi} = \phi/V_s$.
V_s	Potential scale	Data-derived scale stored as `out.scales.V`.
ρ_s	Charge scale	Residual scale stored as `out.scales.rho`.
g_s	Gradient scale	Normal-derivative scale stored as `out.scales.gradPhi`.
$\mathcal{R}$	Poisson residual	$\varepsilon_{\mathrm{Si}}\nabla^2\phi_\theta + \rho$.
$\mathcal{L}_{\mathrm{PDE}}$	PDE loss	Mean squared normalized Poisson residual at interior points.
$\mathcal{L}_{\mathrm{D}}$	Dirichlet loss	Mean squared normalized potential mismatch on Dirichlet boundaries.
$\mathcal{L}_{\mathrm{N}}$	Neumann loss	Mean squared normalized normal-derivative mismatch on Neumann boundaries.
$\mathcal{L}_{\mathrm{data}}$	Data loss	Mean squared normalized potential mismatch at sparse FEM anchors.
w_i	Loss weights	The four dimensionless multipliers applied to PDE, Dirichlet, Neumann, and data losses.

Symbol / acronym	Name	Meaning in this document
CB	Channel-batch layout	`dlarray` label convention with coordinate channels by observations.
Adam	Adaptive moment estimation	Optimizer used for every network update.

1 What the current Module 2 PINN code actually implements

The present code trains one neural network for one fixed electrostatic case:

$$(x, y) \longmapsto \phi_\theta(x, y). \tag{1.1}$$

It solves the constant-permittivity Poisson equation

$$\varepsilon_{\mathrm{Si}} \left(\frac{\partial^2 \phi}{\partial x^2} + \frac{\partial^2 \phi}{\partial y^2} \right) + \rho(x, y) = 0, \qquad \mathbf{E} = -\nabla \phi. \tag{1.2}$$

The network is informed by four possible signals:

$$\mathcal{L}_{\mathrm{total}} = w_{\mathrm{PDE}} \mathcal{L}_{\mathrm{PDE}} + w_D \mathcal{L}_{\mathrm{D}} + w_N \mathcal{L}_{\mathrm{N}} + w_{\mathrm{data}} \mathcal{L}_{\mathrm{data}}. \tag{1.3}$$

The FEM data term is optional. With FEM anchors enabled, the code is a hybrid physics-data PINN. With anchors disabled, it is a pure single-case PINN.

1.1 Current scope versus the eventual surrogate target

The physics note describes three modeling modes. Their status in the executable code is:

Mode	Definition	Current status
Mode A	One pointwise network learns $\phi(x, y)$ for one fixed source and one fixed boundary configuration.	Implemented legacy prototype. Optional sparse FEM anchors may be added.
Mode B	One supervised field operator learns $\boldsymbol{\rho} \mapsto \boldsymbol{\Phi}$ from many complete FEM field pairs.	New target baseline; FEM v2 data contract implemented, neural code not yet implemented.

Mode	Definition	Current status
Mode C	One physics-informed field operator learns $\boldsymbol{\rho} \mapsto \boldsymbol{\Phi}$ using FEM data plus the discrete FEM residual $K\boldsymbol{\Phi}_\theta - \boldsymbol{b}(\boldsymbol{\rho})$ and boundary constraints.	Final target; physics design documented, neural code not yet implemented.

Interpretation rule. The current executable PINN code is an implementation and verification benchmark for pointwise PINN mechanics. It is not the newly selected field-to-field surrogate. Training a new pointwise network for every FEM case may cost more than solving that case directly with FEM; the amortized target is one operator trained over many complete $\boldsymbol{\rho} \to \boldsymbol{\Phi}$ pairs.

1.2 Implemented capabilities

- Two normalized spatial inputs, four default fully connected hidden layers, smooth `tanh` activations, and one normalized potential output.
- First and second coordinate derivatives obtained by nested calls to `dlgradient`.
- Strong-form Poisson residual for uniform scalar silicon permittivity.
- Soft Dirichlet and soft Neumann boundary penalties on randomly sampled edge points.
- Optional sparse FEM nodal anchors selected reproducibly with `randperm`.
- Stochastic resampling of interior and boundary points at every Adam iteration.
- Potential, electric field, dimensional residual, and normalized residual evaluated at arbitrary physical coordinates.
- FEM comparison metrics, analytical checks for three simple cases, seven PNG diagnostics, a text summary, and a saved MAT result.

1.3 Important current limitations

- Physical parameters are selected by `default_module2_params(caseName)`. The top-level PINN driver accepts training options but does not accept arbitrary physical-parameter overrides.
- Every network is tied to one fixed case. Source amplitude, width, center, voltage, geometry, and permittivity are not network inputs.

- Boundary conditions are enforced softly. The predicted boundary value is not exact unless training makes the boundary error sufficiently small.
- The PINN loss can represent a nonzero prescribed normal derivative, but the current reference FEM assembly does not apply nonzero Neumann flux. A nonzero-Neumann custom case would therefore make PINN training targets and FEM validation inconsistent until the FEM boundary integral is implemented.
- The network uses constant scalar permittivity and a strong-form residual. Material interfaces and discontinuous permittivity are not supported.
- The optional data anchors are nodal potential values only. Electric-field FEM values are not directly supervised.
- The optimizer uses a fixed learning rate. There is no scheduler, early stopping, validation split, checkpoint recovery, L-BFGS phase, or adaptive loss weighting.
- The current arrays are created on the CPU. No explicit `gpuArray` path is implemented.
- The smoke test checks callability, sizes, and finite values. It does not enforce a scientific accuracy threshold or prove convergence.

2 Software requirements and first-run preflight

2.1 Required software

The PINN path requires MATLAB and Deep Learning Toolbox. The FEM reference path uses core MATLAB. The driver explicitly checks for:

```
dlarray
dlnetwork
dlgradient
dlfeval
adamupdate
```

No Geant4 executable is called by this Module 2 PINN workflow.

Check the installation from MATLAB:

```
version
ver
license('test','Neural_Network_Toolbox')
which dlarray -all
which dlnetwork -all
```

```
which dlgradient -all
which adamupdate -all
```

The toolbox license identifier retains MATLAB's historical `Neural_Network_Toolbox` name. A value of `1` from `license('test',...)` indicates that MATLAB can check out the license at that moment.

2.2 Correct starting location

Start in the extracted project's MATLAB directory:

```
cd('C:/path/to/RadiationEffects_proj2/matlab')
setup_project_paths
```

The setup function adds `matlab/`, `matlab/cases/`, `matlab/tests/`, `matlab/pinn/`, and all folders below `matlab/src/`.

2.3 Path preflight check

Before training, verify that MATLAB resolves the intended project copy:

```
which main_module2_pinn_electrostatics -all
which module2_pinn_network -all
which module2_pinn_loss -all
which module2_pinn_train -all
which module2_pinn_predict -all
which module2_pinn_make_dataset -all
which module2_pinn_verify -all
which solve_poisson_defect_space_charge_2d -all
which default_module2_params -all
```

Each result should point into the same extracted `RadiationEffects_proj2` tree. If `-all` displays older project copies, remove those paths or restart MATLAB before interpreting any result.

2.4 Verify the FEM dependency first

The dataset generator calls the FEM solver before building anchors. A PINN failure can therefore originate in the reference solver. Run these tests first:

```
test_module2_zero_charge_2d
test_module2_linear_potential_2d
test_module2_uniform_space_charge_2d
```

```
test_module2_localized_defect_charge_2d
```

If any FEM test fails, diagnose it with the Module 2 FEM implementation note before debugging the neural network.

3 Recommended run order

Use the following ladder. Do not begin scientific interpretation with the localized Gaussian case.

1. Confirm file resolution and Deep Learning Toolbox availability.
2. Run the four FEM tests.
3. Run the eight-iteration PINN smoke test.
4. Run a short linear-potential training job to confirm loss movement and file output.
5. Run the full linear-potential case and compare against the analytical solution.
6. Run the uniform-space-charge case to verify second derivatives and the source term.
7. Run the localized-defect case and compare against FEM.
8. Repeat important runs with several random seeds and a controlled hyperparameter study.
9. Perform the FEM-anchor ablation before claiming physics-only performance.

Why this order matters. The linear-potential case isolates boundary enforcement and first derivatives. The uniform-charge case adds nonzero curvature and directly tests the PDE source sign. The Gaussian case is more difficult and has no configured closed-form solution, so FEM is its reference.

4 Quick-start execution recipes

4.1 Small smoke run without files or plots

This is the fastest direct call for confirming that the driver and optimizer execute:

```
setup_project_paths

opts = struct();
opts.maxIterations = 25;
```

```
opts.numInterior = 128;
opts.numBoundaryPerSide = 16;
opts.numDataAnchors = 32;
opts.numHiddenLayers = 2;
opts.numNeurons = 16;
opts.makePlots = false;
opts.writeSummary = false;
opts.saveResults = false;
opts.verbose = true;
opts.printEvery = 5;

out = main_module2_pinn_electrostatics('linear_potential', opts);
```

This run is a plumbing check, not an accuracy result.

4.2 Repository smoke test

```
setup_project_paths
test_module2_pinn_electrostatics
```

The test uses two hidden layers, 16 neurons per layer, 64 interior points, 12 boundary points per side, 32 FEM anchors, and eight Adam iterations.

4.3 Default run

Omitting the case name selects `localized_defect_charge`:

```
setup_project_paths
out = main_module2_pinn_electrostatics;
```

The explicit equivalent is:

```
out = main_module2_pinn_electrostatics( ...
    'localized_defect_charge');
```

4.4 Recommended first full analytical run

```
setup_project_paths
outLinear = main_module2_pinn_electrostatics( ...
    'linear_potential');
```

This uses the driver defaults: four hidden layers, 48 neurons per layer, 1200 iterations, 1024 interior points per iteration, 96 boundary points per side, and 160 FEM anchors.

4.5 Uniform charge and localized defect runs

```
outUniform = main_module2_pinn_electrostatics( ...
    'uniform_space_charge');

outDefect = main_module2_pinn_electrostatics( ...
    'localized_defect_charge');
```

4.6 Pure PINN run without FEM anchors

```
opts = struct();
opts.useDataAnchors = false;
opts.numDataAnchors = 0;
opts.wData = 0.0;
opts.outputDir = fullfile('outputs', ...
    'module2_pinn_2d_pure');

outPure = main_module2_pinn_electrostatics( ...
    'linear_potential', opts);
```

Setting `useDataAnchors=false` makes the data arrays empty. Setting `wData=0` additionally records the intended ablation unambiguously.

4.7 Headless terminal run

```
matlab -batch "cd('C:/path/to/RadiationEffects_proj2/matlab'); setup_project_paths; out
    = main_module2_pinn_electrostatics('linear_potential');"
```

The terminal process returns a nonzero exit status for an uncaught MATLAB error.

5 File map and call graph

File	Responsibility
`matlab/main_module2_pinn_electrostatics.m`	Public driver. Validates requirements and options, seeds randomness, calls all six PINN functions, packages output, and optionally saves the MAT-file.
`matlab/pinn/module2_pinn_make_dataset.m`	Loads one built-in physical case, runs the reference FEM solver, computes scales, and chooses optional nodal anchors.
`matlab/pinn/module2_pinn_network.m`	Builds the smooth two-input, one-output `dlnetwork`.
`matlab/pinn/module2_pinn_train.m`	Resamples collocation and boundary points, calls the loss through `dlfeval`, performs Adam updates, and records component losses.
`matlab/pinn/module2_pinn_loss.m`	Computes PDE, Dirichlet, Neumann, and data losses plus gradients with respect to network parameters.
`matlab/pinn/module2_pinn_predict.m`	Evaluates physical ϕ, $\mathbf{E}$, and residuals at a mesh or arbitrary $N \times 2$ coordinates.
`matlab/pinn/module2_pinn_verify.m`	Computes FEM and analytical metrics, writes seven plots and a summary when enabled.
`matlab/tests/test_module2_pinn_electrostatics.m`	Lightweight regression and finite-value smoke test.
`matlab/src/module2_electrostatics/default_module2_params.m`	Defines the four supported physical cases.
`matlab/src/module2_electrostatics/build_space_charge_module2_2d.m`	Evaluates exactly the same $\rho(x, y)$ used by FEM and PINN training.
`matlab/src/module2_electrostatics/solve_poisson_defect_space_charge_2d.m`	Produces the reference FEM solution used for anchors and validation.

The runtime call sequence is

$$\boxed{\text{driver}} \rightarrow \boxed{\text{FEM dataset and scales}} \rightarrow \boxed{\text{network}} \rightarrow \boxed{\text{training and loss}} \rightarrow \boxed{\text{prediction}} \rightarrow \boxed{\text{verification and save}}.$$

6 Exact execution sequence inside the driver

For `out = main_module2_pinn_electrostatics(caseName,opts)`, MATLAB performs:

1. Select `localized_defect_charge` if `caseName` is absent.
2. Create an empty `opts` structure if options are absent.
3. Add the FEM source, cases, and PINN directories to the path.
4. Require the five essential Deep Learning Toolbox operations.
5. Fill each unspecified option with a default and validate counts, rates, decay factors, and weights.
6. Seed MATLAB's Twister random stream using `opts.randomSeed`.
7. Resolve and create the output directory.
8. Call `module2_pinn_make_dataset`; this runs one complete FEM reference solve.
9. Call `module2_pinn_network` to create an untrained `dlnetwork`.
10. Call `module2_pinn_train`; Adam updates the network for exactly `maxIterations` iterations.
11. Call `module2_pinn_predict` on every FEM mesh node.
12. Call `module2_pinn_verify` for metrics, analytical comparisons, plots, and summary text.
13. Package all results into `out`.
14. Save `out` with MAT version 7.3 when `saveResults=true`.
15. Print the principal errors, residual, time, and output directory.

7 Nondimensionalization and automatic differentiation

7.1 Network input and output

The network receives

$$\hat{x} = \frac{x}{L_x}, \qquad \hat{y} = \frac{y}{L_y}, \tag{7.1}$$

and returns

$$\hat{\phi}_\theta = \frac{\phi_\theta}{V_s}. \tag{7.2}$$

Coordinates therefore normally lie in $[0,1]^2$. The input layer has `Normalization='none'` because normalization is performed explicitly before the network is called.

The `dlarray` layout is

$$\begin{bmatrix} \hat{x}_1 & \cdots & \hat{x}_N \\ \hat{y}_1 & \cdots & \hat{y}_N \end{bmatrix}, \tag{7.3}$$

with label `'CB'`: two channels and N batch observations.

7.2 Scales chosen by the dataset generator

Let $L_s = \max(L_x, L_y)$. The code measures the FEM and boundary potential scale, estimates an equivalent source scale, and then stores

$$\rho_s = \max\left(\max|\rho_{\mathrm{FEM}}|, \frac{\varepsilon_{\mathrm{Si}} V_{\mathrm{estimate}}}{L_s^2}, \texttt{realmin}\right), \tag{7.4}$$

$$V_s = \max\left(\max|\phi_{\mathrm{FEM}}|, \frac{\rho_s L_s^2}{\varepsilon_{\mathrm{Si}}}, 10^{-12}\right), \tag{7.5}$$

$$g_s = \frac{V_s}{L_s}. \tag{7.6}$$

Inspect the realized values rather than assuming them:

```
out.scales
```

7.3 Chain rule used for fields and curvature

Because the network differentiates with respect to normalized coordinates,

$$\frac{\partial \phi}{\partial x} = \frac{V_s}{L_x}\frac{\partial \hat{\phi}}{\partial \hat{x}}, \qquad \frac{\partial \phi}{\partial y} = \frac{V_s}{L_y}\frac{\partial \hat{\phi}}{\partial \hat{y}}, \tag{7.7}$$

$$\frac{\partial^2 \phi}{\partial x^2} = \frac{V_s}{L_x^2}\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2}, \qquad \frac{\partial^2 \phi}{\partial y^2} = \frac{V_s}{L_y^2}\frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2}. \tag{7.8}$$

The code first calls `dlgradient` for $\partial\hat{\phi}/\partial\hat{x}$ and $\partial\hat{\phi}/\partial\hat{y}$, then differentiates those first derivatives again. `'EnableHigherDerivatives',true` is required for this nested calculation.

7.4 Normalized Poisson residual

The physical residual is

$$\mathcal{R} = \varepsilon_{\mathrm{Si}} V_s \left(\frac{1}{L_x^2}\frac{\partial^2 \hat{\phi}}{\partial \hat{x}^2} + \frac{1}{L_y^2}\frac{\partial^2 \hat{\phi}}{\partial \hat{y}^2}\right) + \rho. \tag{7.9}$$

The quantity minimized is $\hat{\mathcal{R}} = \mathcal{R}/\rho_s$, giving

$$\mathcal{L}_{\mathrm{PDE}} = \frac{1}{N_r} \sum_{i=1}^{N_r} \hat{\mathcal{R}}_i^2. \tag{7.10}$$

The dimensional residual returned in `out.pinn.residual` has units of $\mathrm{C\,m^{-3}}$. The normalized residual returned in `out.pinn.residualNormalized` is dimensionless.

8 Loss terms and what each one proves

8.1 Interior PDE loss

At every iteration, `numInterior` new points are drawn uniformly in normalized coordinates. The source builder evaluates physical ρ at those coordinates. A small $\mathcal{L}_{\mathrm{PDE}}$ indicates that the current network approximately satisfies Poisson's equation at the sampled points. It does not independently establish correct boundary values or the unique correct solution.

8.2 Dirichlet loss

For all sides marked `dirichlet`, the code samples `numBoundaryPerSide` points and computes

$$\mathcal{L}_{\mathrm{D}} = \frac{1}{N_D} \sum_i \left(\hat{\phi}_{\theta,i} - \frac{\phi_{D,i}}{V_s} \right)^2. \tag{8.1}$$

This is soft enforcement. Verify boundary errors after training; do not assume a high weight makes them exactly zero.

8.3 Neumann loss

For all sides marked `neumann`, outward normals are constructed explicitly. The code computes

$$\mathcal{L}_{\mathrm{N}} = \frac{1}{N_N} \sum_i \left[\frac{\boldsymbol{n}_i \cdot \nabla \phi_\theta - (\partial\phi/\partial n)_i}{g_s} \right]^2. \tag{8.2}$$

The standard cases use zero normal derivative on top and bottom.

8.4 Sparse FEM-anchor loss

When enabled, the dataset generator selects up to `numDataAnchors` unique FEM mesh nodes once, before training:

$$\mathcal{L}_{\mathrm{data}} = \frac{1}{N_s} \sum_i \left(\hat{\phi}_{\theta,i} - \hat{\phi}_{\mathrm{FEM},i} \right)^2. \tag{8.3}$$

The anchors remain fixed across iterations, while collocation and boundary points are resampled. The selected FEM indices and normalized values are preserved in `out.anchors`.

8.5 Weighted versus unweighted histories

The arrays `history.pde`, `history.dirichlet`, `history.neumann`, and `history.data` store unweighted losses. `history.total` stores their weighted sum. To inspect the actual optimizer competition:

```
h = out.trainingHistory;
o = out.options;

weightedPDE  = o.wPDE       .* h.pde;
weightedDir  = o.wDirichlet .* h.dirichlet;
weightedNeu  = o.wNeumann   .* h.neumann;
weightedData = o.wData      .* h.data;

semilogy(h.iteration, ...
    [weightedPDE, weightedDir, weightedNeu, weightedData])
grid on
legend('weighted PDE','weighted Dirichlet', ...
       'weighted Neumann','weighted data')
```

This plot answers whether one term dominated the objective after weighting.

9 Complete option reference

Only fields explicitly set by the caller replace defaults. All other fields are filled by the driver.

Option	Default	Meaning and diagnostic effect
`randomSeed`	7	Seeds weight initialization, anchor selection, and stochastic collocation for reproducibility.
`numHiddenLayers`	4	Number of fully connected hidden layers. Must be a positive integer.
`numNeurons`	48	Width of every hidden layer. Must be a positive integer.
`maxIterations`	1200	Exact number of Adam updates. There is no early stopping.

Option	Default	Meaning and diagnostic effect
`learnRate`	2.0×10^{-3}	Fixed Adam learning rate. Too large can produce unstable or oscillatory training; too small can stall progress.
`gradientDecayFactor`	0.9	Adam first-moment decay factor. Must satisfy $0 \leq \beta_1 < 1$.
`squaredGradient DecayFactor`	0.999	Adam second-moment decay factor. Must satisfy $0 \leq \beta_2 < 1$.
`numInterior`	1024	Fresh interior collocation points per iteration. Increasing it improves coverage but increases every gradient evaluation.
`numBoundaryPerSide`	96	Fresh points on each active boundary side per iteration. The total depends on how many sides have each boundary type.
`numDataAnchors`	160	Requested unique FEM nodal anchors. The code caps this at the number of FEM nodes.
`useDataAnchors`	`true`	Enables or disables anchor selection.
`wPDE`	1.0	Weight on normalized Poisson residual loss.
`wDirichlet`	20.0	Weight on fixed-potential mismatch. The default emphasizes voltage anchoring.
`wNeumann`	2.0	Weight on normal-derivative mismatch.
`wData`	2.0	Weight on sparse FEM potential anchors.
`verbose`	`true`	Enables periodic console loss reporting.
`printEvery`	100	Iteration interval for progress output; iteration 1 and the final iteration are also printed.
`makePlots`	`true`	Writes seven PNG diagnostics with invisible figures.
`writeSummary`	`true`	Writes a human-readable text summary.
`saveResults`	`true`	Saves the complete `out` structure as MAT version 7.3.
`outputDir`	`outputs/ module2_ pinn_2d`	Relative paths are resolved under `matlab/`; absolute Windows and Unix paths are accepted.

9.1 Safe custom option pattern

```
opts = struct();
opts.randomSeed = 41;
opts.numHiddenLayers = 4;
opts.numNeurons = 64;
opts.maxIterations = 2500;
opts.learnRate = 1.0e-3;
opts.numInterior = 2048;
opts.numBoundaryPerSide = 128;
opts.numDataAnchors = 200;
opts.useDataAnchors = true;
opts.wPDE = 1.0;
opts.wDirichlet = 20.0;
opts.wNeumann = 2.0;
opts.wData = 2.0;
opts.printEvery = 100;
opts.outputDir = fullfile('outputs', ...
    'module2_pinn_linear_seed41');

out = main_module2_pinn_electrostatics( ...
    'linear_potential', opts);
```

Use a distinct output directory for each controlled experiment. Otherwise a rerun of the same case overwrites files with the same deterministic names.

10 Built-in physical cases and expected behavior

Case name	Physical setup	What it verifies
`zero_charge`	$\rho = 0$, left and right potentials both zero.	The exact solution is zero potential and zero field. Useful for finite-value and scale handling, but relative error denominators require care.
`linear_potential`	$\rho = 0$, left potential 0 V, right potential 1 V.	Exact linear potential, uniform E_x, zero E_y, boundary loss, and first derivatives.
`uniform_space_charge`	Uniform $\rho = 5.0 \times 10^{-4}$ C m^{-3}, both contacts at 0 V.	Exact parabolic potential, spatially linear E_x, curvature, source sign, and second derivatives.

Case name	Physical setup	What it verifies
`localized_defect_charge`	Positive Gaussian defect concentration centered in the rectangle, zero contact voltages.	Localized Poisson response and FEM agreement. No simple closed-form rectangular solution is configured.

All four cases use $L_x = 10$ μm, $L_y = 4$ μm, a 41×21 structured node grid split into triangles, and silicon relative permittivity 11.7. The top and bottom boundaries use zero normal derivative.

11 Function-by-function code guide

11.1 Top-level driver

Signature.

```
out = main_module2_pinn_electrostatics(caseName, opts)
```

Use this function for normal runs. Its local helper functions are private to the file: they check toolbox availability, fill defaults, validate options, and recognize absolute output paths.

11.2 Network builder

Signature.

```
net = module2_pinn_network(opts)
```

The builder requires `opts.numHiddenLayers` and `opts.numNeurons`. It constructs

$$(\hat{x}, \hat{y}) \xrightarrow{\text{repeated fully connected + tanh}} \hat{\phi}. \tag{11.1}$$

The final layer is linear. `tanh` is used because the PDE loss needs smooth second derivatives.

Inspect a trained network with:

```
out.net
out.net.Layers
out.net.Learnables
size(out.net.Learnables)
```

11.3 FEM dataset and scale builder

Signature.

```
dataset = module2_pinn_make_dataset(caseName, opts)
```

It calls `default_module2_params`, disables FEM plot and MAT output, runs the FEM solver, computes scales, and selects anchor nodes. It returns `dataset.params`, `dataset.fem`, `dataset.scales`, `dataset.anchors`, and `dataset.caseName`.

This function can be probed independently after supplying the required anchor options:

```
opts = struct('useDataAnchors', true, ...
              'numDataAnchors', 20);
rng(7,'twister')
dataset = module2_pinn_make_dataset( ...
    'linear_potential', opts);

dataset.scales
dataset.anchors.nodeIndices
dataset.fem.params
```

11.4 Training loop

Signature.

```
[net, history, trainingInfo] = ...
    module2_pinn_train(net, dataset, opts)
```

The training function assumes that the driver has already filled all defaults. It preallocates five history arrays, maintains Adam's first and second moment states, resamples a complete batch every iteration, evaluates the loss through `dlfeval`, updates all learnables, and records CPU double scalars.

The local batch sampler is intentionally private. It draws points uniformly, converts them back to meters only for source evaluation, constructs edge normals, and reuses fixed anchors.

11.5 Loss and gradients

Signature.

```
[loss, gradients, terms] = module2_pinn_loss( ...
    net, batch, params, scales, opts)
```

Call it through `dlfeval`; direct ordinary evaluation will not establish the differentiation trace needed by `dlgradient`. The returned `terms` structure contains the four unweighted components. The final call

```
gradients = dlgradient(loss, net.Learnables);
```

differentiates the weighted total loss with respect to all network weights and biases.

11.6 Prediction at a mesh or arbitrary coordinates

Signature.

```
prediction = module2_pinn_predict( ...
    net, query, params, scales)
```

The query can be a structure containing `query.nodes` or an $N \times 2$ numeric array in meters. Example:

```
xLine = linspace(0, out.params.Lx, 201).';
yLine = 0.5*out.params.Ly*ones(size(xLine));
xy = [xLine, yLine];

centerline = module2_pinn_predict( ...
    out.net, xy, out.params, out.scales);

plot(xLine, centerline.phi)
xlabel('x [m]')
ylabel('PINN potential [V]')
grid on
```

The function does not reject coordinates outside the training rectangle. Such values are extrapolations and should not be trusted merely because a finite number is returned.

11.7 Verification function

Signature.

```
verification = module2_pinn_verify( ...
    fem, pinn, history, params, opts, scales, outputDir)
```

Normal users should let the driver call this. It calculates FEM comparison metrics for every case, analytical metrics for `zero_charge`, `linear_potential`, and `uniform_space_charge`, then optionally writes plots and summary text.

12 Returned structure: what to inspect

The top-level output contains:

Field	Meaning
`out.params`	Physical FEM/PINN case structure, including domain, mesh, source, material, and boundary data.
`out.options`	Fully populated training and output options actually used.
`out.scales`	Coordinate, voltage, charge, gradient, and reference-length scales.
`out.net`	Trained `dlnetwork`.
`out.anchors`	Selected FEM node indices and normalized coordinates/potential. Empty in pure-PINN mode.
`out.fem`	Complete reference FEM result, including mesh, ρ, ϕ, and electric field.
`out.pinn`	PINN nodes, ρ, ϕ, field, residual, normalized residual, and extrema.
`out.trainingHistory`	Iteration plus total, PDE, Dirichlet, Neumann, and data loss arrays.
`out.trainingInfo`	Elapsed seconds, number of updates, and final total loss.
`out.metrics`	FEM-relative potential/field errors, absolute errors, residual metrics, and maximum predicted field.
`out.analytic`	Exact arrays and exact-error metrics when supported; otherwise `available=false`.
`out.plotFiles`	Paths to plots written by the verification function.
`out.summaryFile`	Summary-text path, or empty when disabled.
`out.resultFile`	MAT-file path, or empty when saving is disabled.
`out.outputDir`	Resolved absolute or project-relative output directory.

Start an inspection with:

```
out.options
out.scales
out.trainingInfo
out.metrics
out.analytic

```

```
[min(out.pinn.phi), max(out.pinn.phi)]
[min(out.pinn.residualNormalized), ...
 max(out.pinn.residualNormalized)]
```

13 Generated outputs and how to interpret them

With default output flags, files are written under `matlab/outputs/module2_pinn_2d/`. For a case prefix such as `linear_potential`, the actual verification function writes:

- `*_fem_potential_reference.png`;
- `*_pinn_potential.png`;
- `*_pinn_potential_error.png`;
- `*_fem_electric_field_magnitude.png`;
- `*_pinn_electric_field_magnitude.png`;
- `*_pinn_pde_residual.png`;
- `*_pinn_training_loss.png`;
- `*_module2_pinn_summary.txt`;
- `*_module2_pinn_results.mat`.

Inspect them in this order:

1. **Training loss:** check every component, not only the total.
2. **FEM and PINN potential:** confirm sign, scale, boundary behavior, and spatial structure.
3. **Signed potential error:** look for systematic boundary bias, source-localized error, or domain-wide offset.
4. **FEM and PINN field magnitude:** derivative agreement is usually harder than potential agreement.
5. **Residual map:** look for localized PDE violations that an RMS scalar can hide.
6. **Text summary:** confirm the architecture, scales, weights, and metrics match the intended run.
7. **MAT-file:** retain the trained network and all metadata needed to reproduce or probe it.

14 Verification metrics and their limitations

14.1 Potential and electric-field errors

The FEM comparison uses

$$\epsilon_\phi = \frac{\|\phi_{\mathrm{PINN}} - \phi_{\mathrm{FEM}}\|_2}{D_\phi}, \qquad \epsilon_E = \frac{\|\mathbf{E}_{\mathrm{PINN}} - \mathbf{E}_{\mathrm{FEM}}\|_2}{D_E}, \tag{14.1}$$

with scale-aware denominators to keep zero-reference cases finite. Maximum and mean absolute potential error, maximum field-component error, and maximum predicted field are also reported.

14.2 Residual metrics

The reported residual statistics are evaluated on FEM mesh nodes after training:

$$\max_i |\mathcal{R}_i|, \qquad \sqrt{\frac{1}{N}\sum_i \mathcal{R}_i^2}, \qquad \sqrt{\frac{1}{N}\sum_i \hat{\mathcal{R}}_i^2}. \tag{14.2}$$

These nodes are not the same random points used in the final training iteration, so the residual metric is a useful independent spatial check. A stronger study should also evaluate a denser independent grid or random test set.

14.3 No universal pass threshold is encoded

The code intentionally reports errors but does not define a scientific pass threshold. The acceptable error depends on the downstream quantity. For example, peak field near a defect may require a stricter local derivative criterion than a domain-averaged potential. Define acceptance criteria before tuning, record them, and apply the same criteria to all compared models.

15 Repository smoke test: what it checks and what it does not

Run:

```
test_module2_pinn_electrostatics
```

The test first asserts that all six PINN module files are discoverable. If Deep Learning Toolbox is absent, it prints a skip message and returns. If available, it runs a tiny linear-potential job and checks:

- the output, FEM, PINN, training-history, and metric structures exist;

- nodal potential, residual, and electric-field array sizes match the FEM mesh;
- every stored loss, prediction, residual, and selected metric is finite;
- the training history contains the requested eight iterations;
- the summary file exists when summary output is active.

It does *not* prove:

- that eight iterations converge;
- that the final loss is smaller than the initial loss;
- that analytical or FEM errors satisfy a fixed tolerance;
- that the same behavior holds for uniform or Gaussian charge;
- that different random seeds produce consistent results;
- that the PINN is faster than FEM.

The smoke test omits explicit `makePlots`, `writeSummary`, and `saveResults` overrides, so the driver fills their defaults as `true`. It can therefore write normal output files even though its numerical workload is small.

16 Numerical probes after a run

16.1 Check sizes, ranges, and finiteness

```
N = size(out.fem.mesh.nodes,1);

assert(numel(out.pinn.phi) == N)
assert(size(out.pinn.field.Enodal,1) == N)
assert(size(out.pinn.field.Enodal,2) == 2)
assert(all(isfinite(out.pinn.phi)))
assert(all(isfinite(out.pinn.field.Enodal(:))))
assert(all(isfinite(out.pinn.residual)))
assert(all(isfinite(out.trainingHistory.total)))

disp(out.metrics)
```

16.2 Check the analytical solution directly

For a supported case:

```
assert(out.analytic.available)

phiExactError = out.pinn.phi - out.analytic.phi;
fieldExactError = out.pinn.field.Enodal - ...
                  out.analytic.Enodal;

max(abs(phiExactError))
max(abs(fieldExactError),[],'all')
```

16.3 Probe boundary errors independently

```
Ny = 201;
y = linspace(0,out.params.Ly,Ny).';

leftXY  = [zeros(Ny,1), y];
rightXY = [out.params.Lx*ones(Ny,1), y];

leftPred = module2_pinn_predict( ...
    out.net,leftXY,out.params,out.scales);
rightPred = module2_pinn_predict( ...
    out.net,rightXY,out.params,out.scales);

leftError = max(abs(leftPred.phi - ...
    out.params.bc.left.value));
rightError = max(abs(rightPred.phi - ...
    out.params.bc.right.value));

[leftError,rightError]
```

16.4 Probe the Neumann boundary through the field

For zero normal derivative on the bottom and top, $E_y = -\partial\phi/\partial y$ should be near zero:

```
Nx = 201;
x = linspace(0,out.params.Lx,Nx).';

bottomXY = [x, zeros(Nx,1)];
topXY = [x, out.params.Ly*ones(Nx,1)];
```

```
6
7  bottomPred = module2_pinn_predict( ...
8      out.net,bottomXY,out.params,out.scales);
9  topPred = module2_pinn_predict( ...
10     out.net,topXY,out.params,out.scales);
11
12 max(abs(bottomPred.field.Ey_nodal))
13 max(abs(topPred.field.Ey_nodal))
```

16.5 Evaluate an independent dense residual grid

```
1  xv = linspace(0,out.params.Lx,101);
2  yv = linspace(0,out.params.Ly,61);
3  [X,Y] = meshgrid(xv,yv);
4  xyDense = [X(:),Y(:)];
5
6  dense = module2_pinn_predict( ...
7      out.net,xyDense,out.params,out.scales);
8
9  denseRMS = sqrt(mean( ...
10     dense.residualNormalized.^2));
11 denseMax = max(abs(dense.residualNormalized));
12 [denseRMS,denseMax]
```

This includes boundaries. If the goal is an interior-only residual, omit the first and last coordinate rows before forming `xyDense`.

16.6 Inspect anchors spatially

```
1  if ~isempty(out.anchors.nodeIndices)
2      anchorXY = [out.anchors.xHat*out.params.Lx, ...
3                  out.anchors.yHat*out.params.Ly];
4      scatter(anchorXY(:,1),anchorXY(:,2),20, ...
5              out.anchors.phiHat,'filled')
6      axis equal tight
7      colorbar
8  end
```

17 Training convergence and ablation studies

17.1 Do not judge convergence from one total-loss curve

A credible run should show:

- finite and generally decreasing validation errors;
- no single weighted term permanently overwhelming all others;
- small boundary error on a dense independent boundary sample;
- small normalized residual on an independent interior sample;
- stable conclusions across more than one random seed;
- agreement of the actual downstream quantity, such as peak field, not only potential.

Because collocation points change every iteration, component histories can fluctuate even during useful training. Assess trends and independent metrics rather than requiring monotonic loss.

17.2 Minimum controlled experiments

Change one factor at a time and give every run a distinct output directory:

1. iterations: 600, 1200, 2400;
2. interior points: 512, 1024, 2048;
3. network width: 32, 48, 64;
4. FEM anchors: 0, 40, 160;
5. at least three random seeds;
6. one targeted loss-weight sweep after examining weighted loss contributions.

Record training time, potential error, field error, dense residual, boundary error, and peak-field error for every run.

17.3 Anchor ablation

The most informative first ablation compares:

- hybrid training with the default 160 anchors;
- reduced supervision with 40 anchors;

- pure PINN with no anchors.

Keep architecture, random seed, iterations, collocation counts, and loss weights fixed. This isolates how strongly FEM supervision contributes to the result.

There is one implementation-level qualification: the current driver selects FEM anchors before constructing the network. Anchor selection consumes random numbers, so using the same `randomSeed` with different anchor settings does *not* guarantee identical initial network weights. For the current code, compare distributions over several seeds. For a strictly paired ablation, refactor the driver to use separate saved random streams or separate seeds for anchor selection, network initialization, and collocation sampling.

17.4 Loss-weight sweep

Weights do not directly measure physical importance. They condition the optimization. A useful sweep changes one weight over a logarithmic range, then compares independent validation metrics. For example:

```
dirWeights = [2,20,200];
results = cell(size(dirWeights));

for k = 1:numel(dirWeights)
    opts = struct();
    opts.randomSeed = 7;
    opts.wDirichlet = dirWeights(k);
    opts.outputDir = fullfile('outputs', ...
        sprintf('module2_pinn_wD_%g',dirWeights(k)));
    results{k} = main_module2_pinn_electrostatics( ...
        'linear_potential',opts);
end
```

Judge the sweep by boundary, interior, field, and analytical/FEM metrics. The smallest total training loss across differently weighted objectives is not a valid comparison by itself.

18 Reproducibility and timing

The driver calls `rng(opts.randomSeed,'twister')` before the FEM anchors are selected and before the network is built. The same project code, MATLAB numerical environment, options, and seed should therefore reproduce the same random sequence. Exact bitwise reproducibility can still vary across MATLAB releases or hardware math libraries.

For a timed comparison:

```
tic
out = main_module2_pinn_electrostatics( ...
    'linear_potential',opts);
totalWallTime = toc;

trainingTime = out.trainingInfo.elapsedSeconds;
overheadTime = totalWallTime - trainingTime;
```

The stored training time covers only the Adam loop. It excludes the FEM reference solve, network construction, prediction, verification, plotting, summary writing, and MAT-file saving.

For a fair surrogate timing claim, distinguish:

- one-time FEM data generation cost;
- one-time network training cost;
- per-query trained-network evaluation cost;
- per-case FEM solve cost;
- number of future queries over which up-front cost is amortized.

The current single-case implementation does not yet provide the final amortized many-case comparison.

19 Debugging and interactive code tracing

19.1 Stop at the first MATLAB error

```
dbstop if error
out = main_module2_pinn_electrostatics( ...
    'linear_potential');
```

At the prompt, inspect `dbstack`, local variables, array sizes, and the current options. Use `dbquit` to leave debug mode.

19.2 Enter specific stages

```
dbstop in module2_pinn_make_dataset at 1
dbstop in module2_pinn_network at 1
dbstop in module2_pinn_train at 1
```

```
dbstop in module2_pinn_loss at 1
dbstop in module2_pinn_predict at 1
dbstop in module2_pinn_verify at 1
```

The loss function is entered once per iteration, so clear that breakpoint after inspecting one batch:

```
dbclear in module2_pinn_loss
```

19.3 Open and resolve source files

```
open main_module2_pinn_electrostatics
open module2_pinn_train
open module2_pinn_loss
which module2_pinn_loss -all
```

20 Common failures and diagnostic actions

Symptom	Likely cause and next action
Missing `dlarray`, `dlnetwork`, or `adamupdate`	Deep Learning Toolbox is absent, not licensed, or not on the path. Check `ver`, the license test, and `which`.
Undefined PINN or FEM function	`setup_project_paths` was not run, the working copy is incomplete, or an older path shadows the current one. Use `which ... -all`.
Unknown case name	Only the four names in `default_module2_params.m` are accepted. Check spelling and capitalization-independent underscore names.
Loss becomes NaN or Inf	Learning rate may be too large, scales or parameters may be invalid, or gradients may have exploded. Stop at the loss function, inspect each term, reduce the learning rate, and verify `out`-equivalent scales through the dataset stage.
Total loss falls but boundary error remains large	The weighted objective is dominated elsewhere. Plot weighted components and evaluate dense boundary errors; adjust weights only after confirming scaling.
Potential agrees but electric field is poor	Derivatives amplify approximation error. Increase training quality, inspect field and residual maps, and use field-specific acceptance criteria.

Symptom	Likely cause and next action
Residual is small only near training points	Collocation coverage is inadequate or the network overfits sparse locations. Increase and resample points, then evaluate a dense independent residual grid.
Gaussian case converges poorly	The localized source creates higher curvature. Increase collocation density or capacity cautiously and compare against the simpler analytical cases first.
Results change between runs	The random seed or some option changed, a different project copy was resolved, or the MATLAB environment differs. Save `out.options`, archive version, and `which` results.
Output files disappear or change unexpectedly	Reusing the same case and output directory overwrites deterministic filenames. Use one directory per experiment.
Training is slow	Second derivatives are expensive. Reduce points or network size for debugging; disable plots and saving when benchmarking; do not confuse a smoke configuration with the final accuracy configuration.
Out-of-domain prediction looks plausible	The predictor accepts arbitrary coordinates but the network was trained only on the rectangle. Treat out-of-domain values as unsupported extrapolation.
Zero-charge relative errors look surprising	A near-zero reference needs a scale-aware denominator. Inspect absolute errors and residuals as well as relative metrics.

21 Changing physics and adding a new case

The public PINN driver does not accept a physical `params` structure. For a one-off change, the safest current workflow is:

1. create a new named case branch in `default_module2_params.m`;
2. verify that the FEM solver supports the source and boundary configuration;
3. add an FEM test or independent reference;
4. run the PINN through the new case name;
5. extend analytical verification only when a correct closed-form solution exists.

Do not change `out.params` after training and then reuse `out.net`. The network learned the original case even though its input has no case parameter. Changing metadata does not retrain the model.

For repeated custom physical studies, improve the API rather than editing defaults for every run. A future dataset function should accept either a validated `params` structure or a parameter vector $\boldsymbol{\mu}$ and should preserve the exact training-case metadata.

22 Migration target: field-to-field Module 2 surrogate

The next neural-code revision should not add five Gaussian parameters to the old pointwise input. The project decision is instead to learn the complete spatial operator

$$\boxed{\mathcal{G}_\theta : \boldsymbol{\rho} \mapsto \boldsymbol{\Phi}_\theta}. \tag{22.1}$$

The v2 FEM dataset already stores `rho` and `phi` as $N_{\text{nodes}} \times N_{\text{cases}}$ arrays. Gaussian values such as $(C_{\text{peak}}, x_c, z_c, \sigma_x, \sigma_z)$ are stored only under synthetic-generator provenance and must not be used as the canonical neural inputs.

The target implementation therefore requires:

- a field architecture appropriate for either a structured raster (CNN/U-Net/FNO) or the existing triangular mesh (graph/mesh operator);
- complete-field minibatches and complete-field train/validation/test splits;
- normalization of $\boldsymbol{\rho}$ and $\boldsymbol{\Phi}$ using training-set scales only;
- hard or soft treatment of tagged Dirichlet electrode values;
- a differentiable discrete physics residual on free FEM degrees of freedom,

$$\boldsymbol{r}_{\text{PDE}} = K_{FF}\boldsymbol{\Phi}_{\theta,F} + K_{FD}\boldsymbol{\Phi}_D - \boldsymbol{b}_F(\boldsymbol{\rho}); \tag{22.2}$$

- normalized data/PDE/boundary losses followed by adaptive gradient-balanced weighting when needed;
- held-out non-Gaussian fields in addition to Gaussian pilot fields;
- amortized inference-time comparison against FEM;
- saved mesh, normalization, geometry, boundary, and training-domain metadata with the network.

Automatic differentiation will still compute $\nabla_\theta \mathcal{L}$ for backpropagation. The preferred field implementation does not require second derivatives of network output with respect to x and y because the spatial Poisson operator is supplied by the verified FEM matrices.

Do not overstate the current result. A successful run of the files documented in the preceding chapters demonstrates the older pointwise PINN mechanics only. Generalization to arbitrary unseen charge maps and field-to-field surrogate speedup must be demonstrated after the migration above.

23 Recommended verification ladder before using a result

1. **Archive and path:** record the project version and confirm one resolved code tree.
2. **Toolbox:** confirm every required Deep Learning Toolbox operation.
3. **Reference FEM:** pass `test_module2_fem_all_2d`, including the explicit-field and component-field contract tests.
4. **PINN plumbing:** pass the PINN smoke test.
5. **Boundary and first derivative:** validate `linear_potential` analytically.
6. **Curvature and source sign:** validate `uniform_space_charge` analytically.
7. **Localized source:** compare `localized_defect_charge` with FEM.
8. **Independent points:** evaluate dense interior residual and dense boundary errors.
9. **Loss balance:** inspect weighted and unweighted component histories.
10. **Robustness:** repeat with multiple seeds and controlled convergence studies.
11. **Ablation:** compare default anchors, reduced anchors, and pure PINN.
12. **Downstream metric:** verify the quantity that later modules will consume, especially **E** and peak field.

24 Minimal handoff checklist

Before presenting or transferring a Module 2 PINN result, record:

- project archive version, MATLAB version, and Deep Learning Toolbox version;
- physical case name and source/boundary configuration;
- random seed, architecture, optimizer settings, sample counts, anchors, and weights;
- normalization scales from `out.scales`;

- final unweighted and weighted loss components;
- FEM and analytical error metrics where available;
- dense independent residual and boundary checks;
- training time and the scope of that timer;
- repeated-seed or convergence evidence;
- output directory and exact figure/MAT filenames;
- whether the result is hybrid or pure PINN;
- the explicit limitation that the present network represents one fixed case.

25 Code-reading order for a new reader

To understand the implementation without jumping between files randomly, read:

1. `main_module2_pinn_electrostatics.m` through the call sequence and default options;
2. `module2_pinn_make_dataset.m` to understand FEM reference data and scaling;
3. `module2_pinn_network.m` to understand the map $[\hat{x}, \hat{y}] \mapsto \hat{\phi}$;
4. `module2_pinn_train.m` to see stochastic batch construction and Adam;
5. `module2_pinn_loss.m` slowly, following the PDE, boundary, data, and gradient calculations;
6. `module2_pinn_predict.m` to follow the inverse scaling and $\mathbf{E} = -\nabla\phi$;
7. `module2_pinn_verify.m` to see exactly how each metric and output file is constructed;
8. `test_module2_pinn_electrostatics.m` to understand the minimum automated contract.

Final takeaway. The correct workflow is reference first, smallest neural test second, analytical verification third, localized-source comparison fourth, and hyperparameter or architecture tuning last. A trained network is credible only when its boundary behavior, Poisson residual, potential, electric field, reproducibility, and intended downstream metric have all been checked independently.